SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 1 – Scenario-Based Requirement Analysis
Weightage:	20%	Total Marks:	25
CO2 Statement:	Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills		
Student Name:	Kongara Jogesh	Reg. No.:	192525035
Date:	23/9/26	Duration:	60 minutes

Code of Conduct

I K.Jogesh (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: k.jogesh

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the assigned problem statement, prepare a Software Requirement Specification (SRS) document covering the following activities:

1. Study and Analyze the Scenario

- Understand the problem, business domain, stakeholders, challenges and objectives.

2. Identify Functional and Non-Functional Requirements

- List functional requirements and quality requirements including performance, security, reliability, usability, scalability, maintainability and availability.

3. Identify Actors and Use Cases

- Identify primary and secondary actors, list use cases and draw a UML Use Case Diagram.

4. Prepare the SRS

- Include introduction, problem description, scope, requirements, use case descriptions, data, interfaces and system features.

5. Identify Assumptions and Constraints

- Identify assumptions and technical, operational, economic, legal and time constraints.

6. Validate Completeness and Consistency

- Check stakeholder coverage, ambiguity, redundancy, inconsistency and missing requirements.

Assigned Problem Statement – 19. Autonomous Incident Response and Root-Cause Analysis Platform

Industry Context

Complex software systems generate large volumes of logs, metrics, alerts and deployment events. During incidents, engineers must correlate these signals quickly to identify root causes. Alert overload and fragmented observability can delay recovery and increase operational risk.

Problem Statement

Design and develop an Autonomous Incident Response and Root-Cause Analysis Platform that correlates monitoring data, deployment changes, service dependencies and incident histories to identify probable causes and recommend safe remediation.

The platform should integrate application monitoring, centralized logging, incident management, infrastructure-as-code and deployment systems. Automated remediation should operate within predefined safety boundaries and require human approval for high-impact changes.

Objectives

- Correlate metrics, logs, traces and deployment events
- Detect and group related alerts
- Identify probable root causes
- Recommend appropriate remediation actions
- Automate low-risk recovery within predefined limits
- Maintain incident histories and audit trails
- Support human approval for high-impact actions

Expected Outcomes

- Reduce alert overload
- Improve root-cause identification
- Reduce incident response and recovery time
- Improve production-service reliability
- Reduce repetitive operational effort
- Provide auditable and controlled automated incident response

SOFTWARE REQUIREMENT SPECIFICATION (SRS)

1. Introduction

1.1 Purpose

This SRS defines scope, stakeholders, functional behaviour, quality requirements, interfaces, data entities, constraints and validation criteria. It creates a shared baseline for engineers, operations, security and testers.

1.2 Scope

The platform collects telemetry, correlates alerts with service dependencies and deployment events, ranks probable causes, recommends remediation and enforces risk-based execution controls. It stores incident history and audit evidence. The first release is a controlled prototype, not an unrestricted production automation system.

1.3 Terms and Acronyms

Term	Meaning
Incident	Unplanned interruption or degradation of a service.
Telemetry	Metrics, logs, traces and events.
RCA	Root-Cause Analysis.
MTTD	Mean Time to Detect.
MTTR	Mean Time to Resolve/Recover.
RBAC	Role-Based Access Control.
IaC	Infrastructure as Code.

2. Scenario Analysis and Problem Decomposition

2.1 Current Challenges

Signals are often distributed across separate tools. Engineers must search logs, compare timestamps, inspect deployment records, understand dependencies and review prior incidents manually. Duplicate alerts increase cognitive load, while a nearby deployment may be a clue but is not proof of causality.

Challenge	Operational effect	Requirement response
Duplicate alerts	Alert fatigue	Deduplicate and correlate
Fragmented evidence	Slow diagnosis	Normalized events and timeline
Hidden dependencies	Incorrect service blamed	Service context
Deployment uncertainty	Regression may be missed	Release correlation
Unsafe automation	Production impact	Risk policy and approval
Incomplete audit	Weak accountability	Action and decision history

2.2 Stakeholders

Stakeholder	Need	Influence
SRE / DevOps	Fast diagnosis and safe recovery	High
Incident Manager	Ownership, timeline, escalation	High
Application Developer	Logs, traces and deployment evidence	Medium
Security Administrator	Least privilege and audit	High
Cloud/Platform Team	Infrastructure status and controlled actions	High
Product Owner / Management	Lower downtime and measurable KPIs	High
Auditor	Traceable decisions	Medium

2.3 Success Measures

Objective	Metric	Desired direction
Reduce alert overload	Duplicate alerts per incident	Decrease
Improve RCA	Top-ranked cause accepted in review	Increase
Faster detection	MTTD	Decrease
Faster recovery	MTTR	Decrease
Reliability	Availability / recurrence rate	Improve
Auditability	Actions with complete records	Approach 100%

3. Scope and Architecture

3.1 Included Features

- Telemetry ingestion and normalization
- Alert grouping and incident timeline
- Service/deployment correlation
- Probable cause ranking with evidence
- Risk classification and remediation recommendations
- Bounded low-risk execution
- Human approval for high-impact actions
- Incident history, audit and dashboard

3.2 Excluded from Initial Prototype

- Unrestricted production changes
- Replacing all existing monitoring tools
- Guaranteeing a cause when evidence is ambiguous
- Destructive actions without approval

3.3 High-Level Architecture

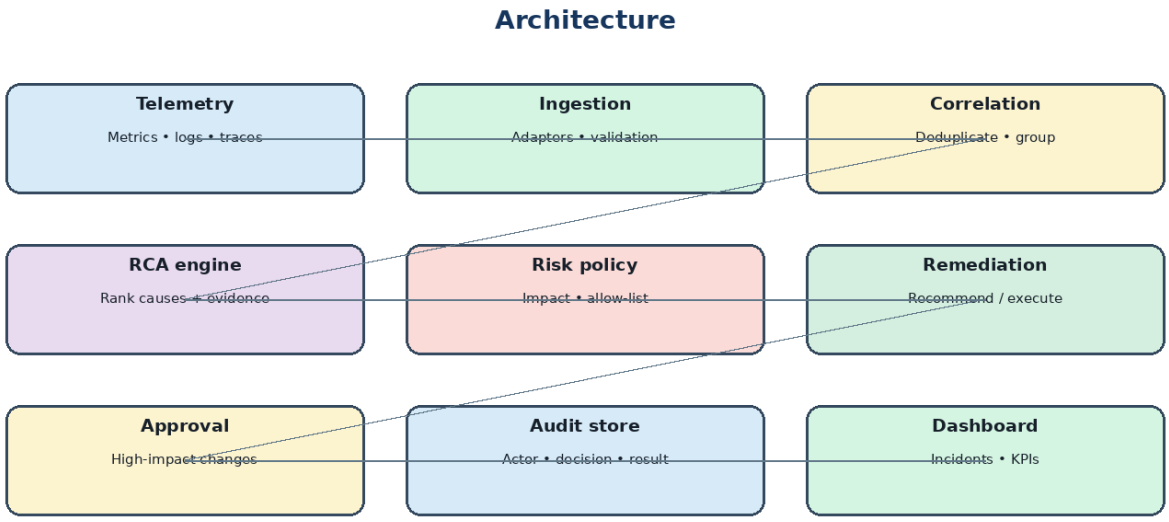


Figure 1. New component architecture created for this assessment.

The architecture separates ingestion, correlation, root-cause analysis, policy, remediation, approval, audit and dashboard components. This supports modular testing and scaling.

3.4 Data Flow

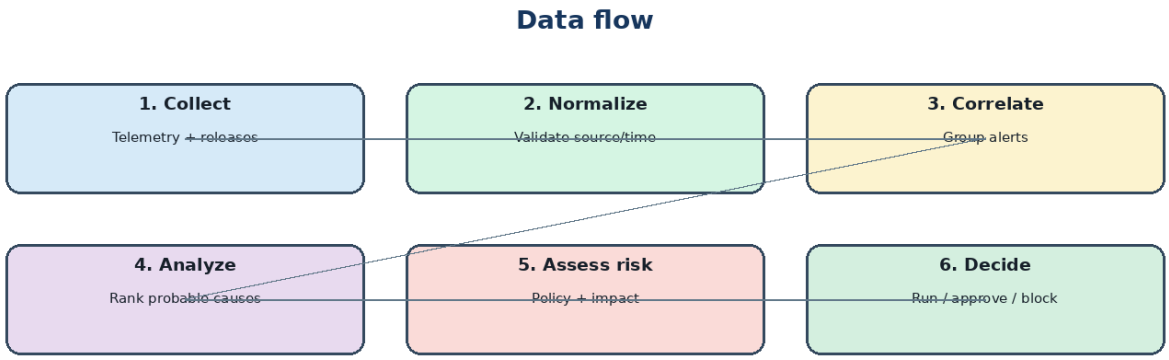


Figure 2. Incident data-flow diagram.

Events are validated before correlation. The platform builds incident context, ranks possible causes, evaluates policy and then executes, requests approval or blocks. Recovery is verified before closure.

4. Functional Requirements

Each requirement has a unique identifier and an acceptance check.

ID	Functional requirement	Acceptance evidence
FR-01	Ingest metrics, logs, traces and deployment events.	Source and timestamp are retained.
FR-02	Validate and normalize incoming events.	Invalid events are quarantined with reason.
FR-03	Group related alerts by time, service and signal similarity.	Known related alerts form one incident.
FR-04	Maintain chronological incident timeline.	Timeline orders alerts, releases and actions.
FR-05	Identify affected services using dependency data.	Incident lists linked services.
FR-06	Rank probable causes using evidence and history.	Rank, confidence and evidence displayed.
FR-07	Correlate deployment events without assuming causality.	Related deployment shown as evidence.
FR-08	Recommend remediation for supported incident types.	Recommendation has rationale and risk.
FR-09	Classify actions low, medium, high or blocked.	Policy tests match expected risk.
FR-10	Execute only allow-listed low-risk actions when enabled.	Disallowed action is never run.
FR-11	Require approval for high-impact actions.	Execution blocked before approval.
FR-12	Record policy decisions, approvals and results.	Audit query returns action chain.
FR-13	Notify owners when intervention is required.	Delivery or failure is recorded.
FR-14	Control incident state transitions.	Invalid transition is rejected.
FR-15	Display incidents, severity, MTTD, MTTR and approvals.	Dashboard matches stored data.
FR-16	Enforce role-based access.	Unauthorized requests denied.
FR-17	Allow authorized user to disable automation.	Disable control stops automatic actions.
FR-18	Apply configurable retention rules.	Retention policy is enforced.

5. Non-Functional Requirements

ID	Quality requirement	Verification
NFR-01 Performance	Prototype event available for correlation within 5 seconds under defined load.	Measure latency.
NFR-02 Availability	Production design target 99.5% monthly, excluding agreed maintenance.	Availability report.
NFR-03 Security	Authenticate users and authorize privileged actions.	Negative access tests.
NFR-04 Reliability	Connectors retry with bounded backoff and show health.	Simulated outage.
NFR-05 Scalability	Workers support horizontal scaling and partitioning.	Increasing-load test.
NFR-06 Maintainability	Connectors and policies modular and documented.	Code review.
NFR-07 Usability	Operator can locate incident and timeline quickly.	Usability walkthrough.
NFR-08 Auditability	Record identity, time, policy result and outcome.	Audit completeness.
NFR-09 Safety	High-impact action cannot run without approval.	Policy integration test.
NFR-10 Interoperability	Use documented APIs, webhooks or adapters.	Contract test.
NFR-11 Privacy	Secrets excluded from logs; access follows policy.	Secret scan.
NFR-12 Recoverability	Restart preserves persisted incident/audit records.	Restart test.

5.1 Quality Trade-offs

Aggressive correlation can reduce alert noise but may merge unrelated incidents. Automation can reduce recovery time but can also amplify mistakes. The platform therefore exposes evidence and uncertainty and keeps high-impact actions behind explicit approval.

5.2 Role/Permission Matrix

Role	View	Approve high-impact	Change policy	Manage users
Operator	Yes	If granted	No	No
Incident Manager	Yes	If authorized	No	No
Security Admin	Yes	If authorized	Yes	Yes
Auditor	Read-only	No	No	No

6. Actors and Use Cases

Actor	Type	Responsibilities
SRE / DevOps	Primary	Inspect evidence, review causes and run permitted actions.
Incident Manager	Primary	Assign ownership and approve actions if authorized.
Security Administrator	Primary	Manage roles, policy and safety boundaries.
Monitoring/logging systems	External	Send alerts, metrics, logs and traces.
Deployment/CI-CD	External	Send release metadata.
Incident management tool	External	Synchronize tickets and status.
Auditor	Secondary	Review action and approval history.

6.1 Use Case Overview

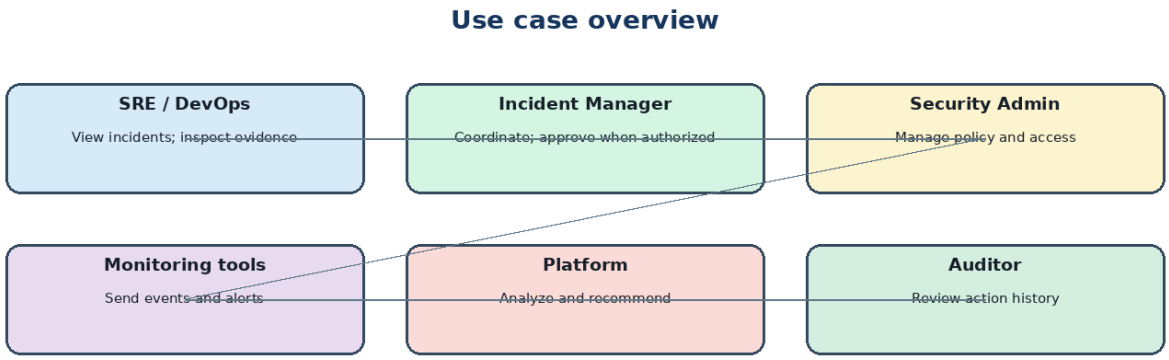


Figure 3. Actor and use-case overview created for this assessment.

Use case	Actor	Precondition	Result
Ingest telemetry	Monitoring tool	Connector configured	Event normalized or quarantined.
Correlate alerts	Platform/SRE	Valid events available	Incident created or updated.
Analyze root cause	SRE	Evidence exists	Ranked causes displayed.
Recommend action	SRE	Cause/action mapping exists	Action with risk and rationale.
Approve action	Authorized approver	High-impact action pending	Decision recorded; rejection blocks execution.
Execute low-risk action	Automation	Allow-listed and enabled	Result and health check recorded.
Review audit	Auditor	Authorized access	Read-only action history.

7. External Interfaces and Data Requirements

Interface	Data exchanged	Requirement
Monitoring	Metrics, alerts, API/webhook	Authenticate source and preserve timestamps.
Logging	Structured log events	Validate schema and redact secrets.
Tracing	Trace/span IDs	Link events when IDs exist.
Deployment/CI-CD	Release ID, commit, service, environment, time	Display release as evidence.
Incident manager	Ticket ID, severity, owner, state	Avoid duplicate ticket creation.
IaC	Plan/apply metadata	Treat production changes as high impact unless policy permits.
Notifications	Email/chat/webhook	Record delivery results.
Identity provider	Identity and role	Authorize approvals.

7.1 Logical Data Model

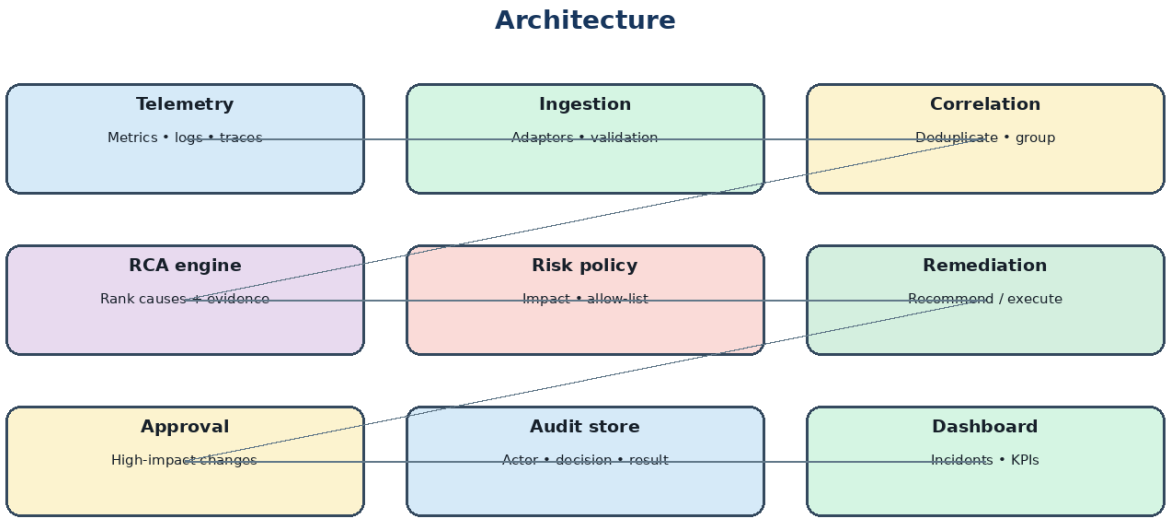


Figure 4. System component relationships supporting the logical entities.

Entity	Key attributes	Purpose
INCIDENT	incident_id, severity, status, start_time	Incident lifecycle.
EVENT	event_id, source, timestamp, type, service_id	Correlated evidence.
SERVICE	service_id, name, criticality, dependencies	Affected service context.
ROOT_CAUSE	cause_id, incident_id, summary, confidence	Probable causes and evidence.
ACTION	action_id, incident_id, risk_level, status	Remediation decision.
APPROVAL	approval_id, action_id, approver, decision	Human decision.
AUDIT	audit_id, actor, timestamp, outcome	Governance record.

7.2 Data Quality and Retention

- Use UTC internally; render local time in the UI.
- Preserve source, environment and service IDs.
- Temporal correlation alone is not proof of causality.
- Retain evidence used for each RCA result.
- Use append-only audit records for normal users.
- Define retention and redaction with security stakeholders.

8. Incident Handling Sequence

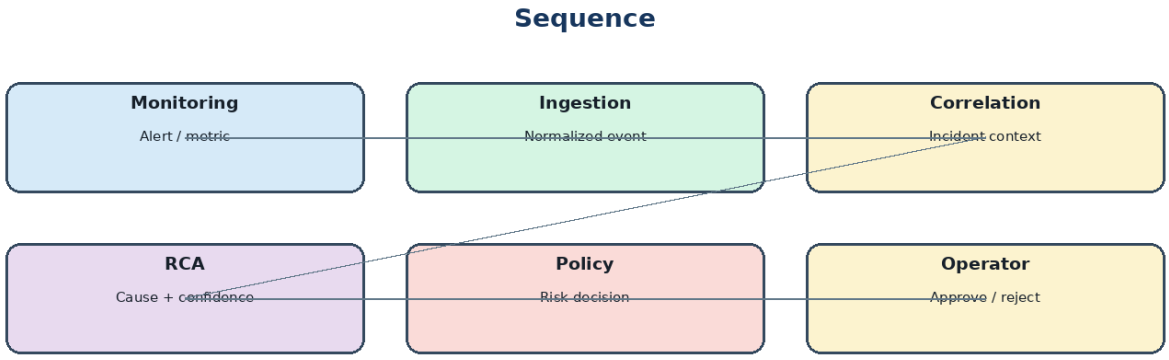


Figure 5. New incident-handling sequence view.

This is a designed sequence illustration, not a screenshot from a live production system. It shows the intended information flow among monitoring, ingestion, correlation, RCA, policy evaluation, approval and verification.

8.1 Incident State Model

State	Meaning	Next states
New	Incident candidate created	Investigating / duplicate closed
Investigating	Evidence gathered and causes ranked	Awaiting approval / mitigating / monitoring
Awaiting approval	High-impact action pending	Mitigating / investigating / closed
Mitigating	Permitted action executing	Monitoring / investigating
Monitoring	Recovery checks running	Closed / investigating
Closed	Resolution and audit saved	Reopen if recurrence detected

8.2 Exception Handling

- If a source is unavailable, mark it unhealthy and continue with available evidence.
- Quarantine events with invalid timestamps.
- If RCA confidence is low, avoid automatic remediation.
- Stop retries after configured timeout and escalate.
- If approval is rejected or expires, do not execute.
- If health checks fail, reopen investigation.

9. Dashboard and Illustrative Output

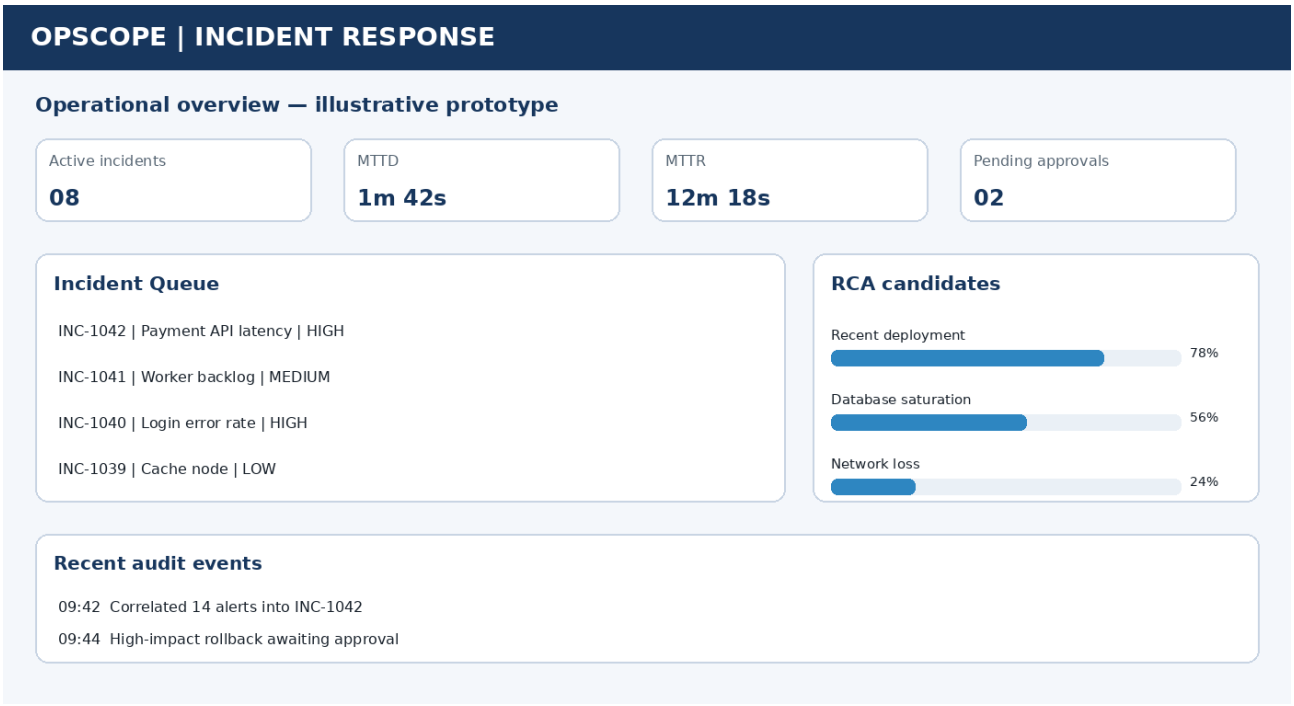


Figure 6. Dashboard prototype mock-up; figures are sample demonstration data, not measured production results.

The dashboard demonstrates active incidents, MTTD, MTTR, pending approvals, incident queue, probable-cause candidates and recent audit events. A genuine application output screenshot should be captured after the software prototype is implemented and run; this designed screen must not be described as a live execution capture.

Field	Illustrative incident record
Incident ID	INC-1042
Service	Payment API
Severity	High
Symptoms	Latency spike and elevated error rate
Evidence	14 correlated alerts; recent deployment near incident onset
Top candidate	Recent deployment (78% illustrative confidence)
Recommendation	Pause rollout; route rollback for approval
Policy result	High impact — approval required
Status	Awaiting authorized approval

9.1 Sample Audit Events

Time	Actor	Event	Result
09:42:10	Correlation engine	Grouped 14 alerts into INC-1042	Incident updated
09:43:02	RCA service	Ranked recent deployment	Evidence displayed
09:44:15	Policy engine	Evaluated rollback request	Approval required
09:45:00	Incident manager	Opened approval request	Pending decision

9.2 KPI Definitions

KPI	Definition	Purpose
MTTD	Fault onset to detection	Detection performance
MTTR	Incident start to recovery	Recovery efficiency
Alert grouping ratio	Raw alerts / incident groups	Noise reduction
RCA top-1 accuracy	Top candidate accepted in review	Analysis quality
Automation success	Successful authorized actions / attempts	Automation reliability
Approval delay	Request to decision duration	Workflow bottleneck

10. Risk-Based Remediation and Safety Controls

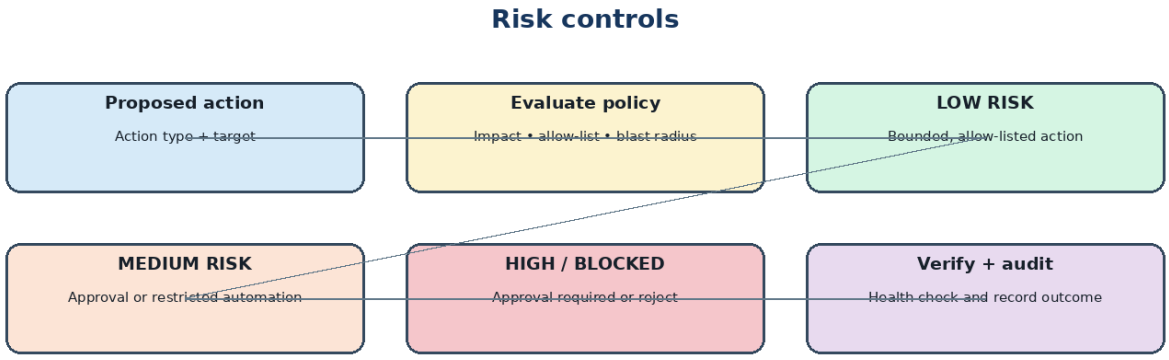


Figure 7. Risk decision path with allow, approval and block outcomes.

Risk	Example	Execution rule
Low	Restart stateless worker within limits	May auto-execute if allow-listed and enabled.
Medium	Scale capacity or restart shared dependency	Approval unless explicit policy permits.
High	Database rollback or critical infrastructure change	Authorized human approval required.
Blocked	Outside boundary or disallowed target	Never execute automatically.

10.1 Mandatory Controls

- Allow-list action types and target resources.
- Use least-privilege service accounts.
- Set retries, timeouts, concurrency and blast-radius limits.
- Require authorized approval for high-impact changes.
- Provide global kill switch and per-service disable.
- Run health checks after actions.
- Use rollback where safe.
- Audit successful, failed, rejected and blocked attempts.

11. Assumptions and Constraints

11.1 Assumptions

- External tools expose supported APIs or webhooks.
- Service identifiers can be mapped across sources.
- Historical incidents are available subject to permissions.
- Operations staff can define allowed actions and risk rules.
- Approvers are available for high-impact actions.
- Initial prototype uses sample/non-production integrations unless approved.

11.2 Constraints

Type	Constraint	Design implication
Technical	Different schemas and connector limits	Adapters and normalization.
Operational	High-impact changes need approval	Approval workflow.
Security	Least privilege and auditability	RBAC and append-only audit.
Economic	Student project budget/time	Core vertical slice first.
Legal/policy	Retention and access policy	Configurable retention/redaction.
Schedule	Assessment timeline	Defer advanced ML and broad integrations.
Reliability	External systems can fail	Retries and graceful degradation.

11.3 Design Implications

Build reliable ingestion, explainable RCA evidence, safe action policy and audit logging before expanding autonomous capabilities. A small trustworthy prototype is preferable to a broad demo that cannot explain or control its actions.

12. Requirement Validation and Traceability

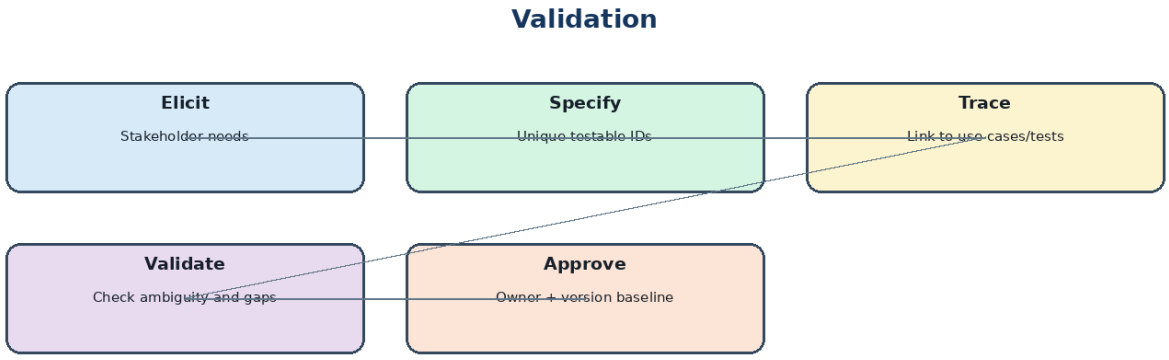


Figure 8. Requirement validation cycle.

Check	Question	Evidence
Completeness	Are objectives represented?	FR-01–FR-18, NFRs and KPI table.
Consistency	Do requirements conflict?	High-impact actions remain approval-gated.
Unambiguity	Can statements be interpreted clearly?	Unique IDs and shall statements.
Verifiability	Can requirements be tested?	Acceptance evidence and test plan.
Traceability	Can requirements link to use cases?	IDs and traceability matrix.
Feasibility	Can prototype be delivered?	Bounded scope and staged plan.
Security	Are privileged actions protected?	RBAC, allow-list and approval.
Failure handling	Are outages covered?	Exception rules.

12.1 Traceability Matrix

Objective	Requirement IDs	Use case	Evidence
Correlate signals	FR-01–FR-04	UC-01/02	Normalization and grouping tests.
Use service/deployment context	FR-04/05/07	UC-02/03	Timeline includes linked events.
Identify probable cause	FR-06	UC-03	Ranked candidate and evidence.
Recommend safe action	FR-08/09	UC-04	Rationale and risk result.
Bounded automation	FR-10/17	UC-06	Allow-list and kill-switch tests.
Human approval	FR-11	UC-05	Unapproved action denied.
Audit history	FR-12/18	UC-07	Complete audit record.
Operational visibility	FR-13–FR-16	UC-02/07	Notifications, status, dashboard.

12.2 Improvements Before Production

- Measure actual event volume and peak ingestion rate.
- Agree service-level objectives with stakeholders.
- Calibrate RCA confidence against reviewed incidents.
- Specify authentication and retry policy for every connector.
- Approve retention and redaction rules with security.
- Build test data for duplicates, deployment regressions and dependency failures.

13. Testing and Acceptance Plan

Test	Scenario	Pass condition
Unit	Event missing service ID	Quarantined with reason.
Integration	Monitoring webhook sends alert	Event stored with source/time.
Correlation	Send 10 related alerts	One incident candidate.
RCA	Use known incident evidence	Expected cause appears in ranked list.
Safety	High-impact action without approval	Action remains unexecuted.
Authorization	Operator attempts policy change	Denied and audited.
Recovery	Connector outage then restoration	Failure visible; connector recovers.
Audit	Approve and reject actions	Both outcomes recorded.
Performance	Replay defined event load	Latency meets agreed target.

13.1 Acceptance Scenario

Given a payment service emits elevated latency alerts and a deployment occurred shortly before the incident, when the platform ingests and correlates the signals, then it creates or updates one incident, shows the deployment as evidence, ranks probable causes, recommends a risk-classified action, requires approval for a high-impact rollback, and records the decision and outcome.

13.2 Release Checklist

- Critical functional requirements pass tests.
- No high-impact action bypasses approval.
- Audit covers success, rejection and failure paths.
- RCA shows evidence and uncertainty.
- Connector failures are visible and recoverable.
- Dashboard values match stored data.
- Limitations and operator procedures are documented.

14. Conclusion and Rubric Mapping

The scenario describes an operational problem in which evidence for incident diagnosis is distributed across monitoring, logging, deployment and service-management tools. This SRS turns the scenario into testable requirements and specifies actors, interfaces, data, constraints and validation methods.

The solution follows a controlled response path: ingest signals, correlate alerts, build context, rank causes, recommend remediation, evaluate risk, request approval where needed, execute only permitted actions, verify recovery and record the outcome. Temporal correlation is treated as evidence rather than proof of causality, and high-impact actions cannot run without authorization.

Unique requirement IDs, use-case descriptions, verification evidence, traceability tables and acceptance tests support completeness and consistency. The recommended implementation order is ingestion and correlation, incident context and RCA, then policy-controlled remediation, approval, audit and dashboard. MTTD, MTTR, alert noise, RCA top-1 accuracy and audit completeness can be used to evaluate success.

Rubric criterion	Covered sections
Scenario understanding and problem analysis (30%)	Sections 1–2
Functional and non-functional requirements (25%)	Sections 4–5
Actor identification and use case modelling (20%)	Section 6
SRS documentation (15%)	Sections 1–10
Assumptions, constraints and validation (10%)	Sections 11–13

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Thorough understanding of scenario, stakeholders, objectives and business context.	Good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing.
Functional and Non-Functional Requirement Identification (25%)	Comprehensive, accurate and well-classified requirements.	Most requirements identified; minor errors.	Basic requirements; omissions.	Incomplete or poorly classified.
Actor Identification and Use Case Modelling (20%)	All actors/use cases; accurate UML diagram.	Most actors/use cases; minor errors.	Limited actors/use cases; partial diagram.	Missing or inaccurate.
SRS Documentation (15%)	Complete, professional SRS with required sections.	Organized with minor gaps.	Essential sections with limited detail.	Incomplete or poorly organized.
Assumptions, Constraints and Requirement Validation (10%)	Realistic assumptions/constraints; validates completeness and consistency.	Most assumptions/constraints; adequate validation.	Limited assumptions; superficial validation.	Missing or incorrect.

Criteria	Mark
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
SRS Documentation (15%)	/5
Assumptions, Constraints and Requirement Validation (10%)	/5
TOTAL	/25

15. Detailed Requirement Specifications and Business Rules

This section expands the high-level requirements into observable system behaviour. Thresholds are prototype defaults and must be calibrated with actual service data.

15.1 Event Ingestion and Normalization

Rule ID	Detailed requirement	Validation / response
BR-01	Every accepted event contains a source identifier and timestamp.	Quarantine missing or invalid timestamps and record reason.
BR-02	Events should include service/resource identifier when available.	Unmapped events remain searchable and marked unlinked.
BR-03	Repeated external events must not create unlimited duplicates.	Use source ID or fingerprint within a time window.
BR-04	Connector failures must be visible.	Mark unhealthy and retry with bounded backoff.
BR-05	Secrets must not appear in incident summaries.	Redact known secret patterns before persistence.

15.2 Correlation and Root-Cause Analysis Rules

Correlation should combine multiple evidence types rather than rely on one signal. Evidence can include temporal proximity, service identity, dependency relationships, deployment timing, trace links and similar historical incidents. The platform should show supporting evidence and limitations.

Evidence	Example	Use	Caution
Temporal	Latency rises after release	Prioritize release for review	Timing is not proof
Service identity	Alerts name payment-api	Group service signals	Aliases may be wrong
Dependency	Payment API calls saturated DB	Expand investigation	Graph may be stale
History	Similar past incident	Add context	Past case may differ
Deployment	Release near onset	Show candidate evidence	May be coincidental

15.3 Remediation Business Rules

- BR-06: Every proposed action identifies target, reason, risk, expected effect and verification check.
- BR-07: Only allow-listed actions may be automated.
- BR-08: High-impact actions require an authorized approver.
- BR-09: Rejected, expired or revoked approval leaves the action unexecuted.
- BR-10: Failed post-action health checks reopen investigation and record failure.
- BR-11: Global automation-off overrides service-level settings.
- BR-12: The policy version used for each decision is recorded.

16. Detailed Use Case Specifications

16.1 UC-02: Correlate Alerts into an Incident

Item	Description
Primary actor	Platform correlation service; SRE reviews the result.
Trigger	A new normalized event arrives.
Preconditions	Connector configured; event passes schema validation.
Main flow	Read service/source/time/type; search open incidents; compare service, fingerprints and signal similarity; link to matching incident or create one; update timeline; notify owner when required.
Alternative flow	If service mapping is missing, keep event unlinked for triage. If storage is unavailable, queue the event and raise a health warning.
Postconditions	Incident/event relationship is persisted and traceable.
Acceptance criteria	Related events are grouped; unrelated services are not merged solely due to timing.

16.2 UC-03: Analyze Probable Root Cause

Item	Description
Primary actor	SRE / DevOps engineer.
Trigger	Incident is created or new evidence arrives.
Preconditions	Incident contains valid events.
Main flow	Collect events; load dependencies and deployment metadata; extract features; generate candidates; rank candidates; store score/evidence; display uncertainty.
Alternative flow	If evidence is sparse, return insufficient evidence. If a connector is unavailable, mark evidence incomplete.
Postconditions	A versioned RCA result is available for review.
Acceptance criteria	Every candidate has a reason; low confidence is labelled uncertain.

16.3 UC-05: Approve a High-Impact Action

Item	Description
Primary actor	Authorized incident manager or approved approver.
Trigger	Policy classifies a proposed action as high impact.
Preconditions	Valid action/target and approver permission.
Main flow	Create request; show incident context; review risk/evidence; accept or reject; re-check approval; execute only after acceptance and policy re-check; record outcome.
Alternative flow	Rejected or expired requests remain blocked. Changed incident context requires re-evaluation.
Postconditions	Approval and action outcome are auditable.
Acceptance criteria	No high-impact action runs before valid approval.

17. Test Cases with Expected Results

These are proposed tests for the prototype, not claims that a production deployment has already passed them.

Test ID	Input / condition	Expected result	Priority
TC-01	Valid alert with source, timestamp and service	Accepted and normalized	High
TC-02	Missing timestamp	Quarantined with reason	High
TC-03	Five duplicate alerts in same fingerprint window	Grouped according to policy	High
TC-04	Similar timing but unrelated services	Not merged solely by time	High
TC-05	Deployment near latency increase	Shown as evidence, not proof	High
TC-06	Insufficient RCA evidence	Uncertainty shown	High
TC-07	Allow-listed low-risk action; automation enabled	Can proceed within limits	High
TC-08	High-impact rollback without approval	Execution blocked	Critical
TC-09	Approval rejected	Action unexecuted; decision audited	Critical
TC-10	Post-action health check fails	Investigation reopens	High
TC-11	Unauthorized policy edit	Denied and recorded	Critical
TC-12	Auditor reads history	Read-only history returned	Medium
TC-13	Connector outage	Health status and bounded retries	High
TC-14	Global automation disabled	No automatic action executes	Critical
TC-15	Restart after incident stored	Incident and audit persist	High

17.1 Acceptance and Exit Criteria

- All critical safety tests pass before automation is enabled.
- High-priority functional tests pass or have a documented exception.
- Every requirement links to at least one verification activity.
- Test records include ID, input, expected result, actual result and pass/fail.
- Known limitations are documented for operators.

18. Implementation Plan and Prioritization

A staged implementation allows validation of the core workflow before remediation is enabled.

Phase	Work items	Deliverable	Exit condition
1. Foundation	Repository, schema, sample event generator	Project skeleton	Build and basic tests pass
2. Ingestion	Adapter, validation, normalization	Normalized event store	Valid/invalid event tests pass
3. Correlation	Fingerprint, time window, service mapping	Incident grouping	Duplicate/unrelated-event tests pass
4. RCA	Evidence extraction and ranking	Ranked cause view	Candidates show evidence
5. Safety	Risk policy, allow-list, approval	Controlled action workflow	Approval cannot be bypassed
6. Verification	Health checks, retries, failure paths	Recovery verification	Failure reopens incident
7. Audit/UI	Dashboard, action history, KPIs	Demo dashboard	UI matches stored data
8. Validation	Tests, security review, report	Evidence pack	Acceptance checklist reviewed

18.1 Priority Classification

Priority	Feature	Reason
P0 – Must	Validation, grouping, safety policy, audit	Trustworthy end-to-end demonstration
P1 – Should	RCA evidence ranking, approvals, dashboard	Diagnosis and operator workflow
P2 – Could	More connectors, historical similarity	Expand after core reliability
Out of scope initially	Unrestricted production changes	Unsafe without mature governance

18.2 Definition of Done

- Code reviewed and documented.
- Unit and integration tests executed and results saved.
- Safety and authorization tests pass.
- Errors logged without exposing secrets.
- Recommendations explain rationale and risk.
- Audit records actor/service, time, policy and outcome.
- Screenshots captured from the actual running prototype.

19. Prototype Code and Real Execution Evidence

The Python example demonstrates service-based event grouping, simple evidence-based cause ranking and risk classification. It uses hard-coded sample events so the behaviour is reproducible without external credentials. Scores are illustrative rules, not trained-model probabilities.

19.1 Python Demonstration Code

```
events = [
    {"time": "09:41:10", "service": "payment-api", "type": "latency", "value": 920},
    {"time": "09:41:18", "service": "payment-api", "type": "http_5xx", "value": 12},
    {"time": "09:41:30", "service": "payment-api", "type": "deployment", "value": "release-2.4.1"},
    {"time": "09:42:05", "service": "payment-api", "type": "latency", "value": 980},
    {"time": "09:42:12", "service": "worker-service", "type": "queue_depth", "value": 45},
]
def correlate(events):
    grouped = {}
    for event in events:
        grouped.setdefault(event["service"], []).append(event)
    return grouped
def rank_causes(service_events):
    types = {e["type"] for e in service_events}
    candidates = []
    if "deployment" in types and ("latency" in types or "http_5xx" in types):
        candidates.append(("Recent deployment", 78, "Deployment and degradation overlap"))
    if "latency" in types:
        candidates.append(("Capacity or database pressure", 56, "Elevated latency observed"))
    if "http_5xx" in types:
        candidates.append(("Application error regression", 42, "HTTP 5xx events observed"))
    return sorted(candidates, key=lambda x: x[1], reverse=True)
def risk_decision(action):
    rules = {
        "restart_stateless_worker": ("LOW", "May run only if allow-listed and enabled"),
        "scale_shared_database": ("HIGH", "Human approval required"),
        "rollback_production": ("HIGH", "Human approval required"),
    }
    return rules.get(action, ("BLOCKED", "Action not allow-listed"))
groups = correlate(events)
print("AUTONOMOUS INCIDENT RESPONSE - DEMO OUTPUT")
print("Events received:", len(events))
for service, service_events in groups.items():
    print(f"\nService: {service} | Events: {len(service_events)}")
    for i, (name, confidence, evidence) in enumerate(rank_causes(service_events), 1):
        print(f"  RCA candidate {i}: {name} ({confidence}% illustrative score)")
        print(f"    Evidence: {evidence}")
print("\nPolicy check: rollback_production")
risk, note = risk_decision("rollback_production")
print(f"  Risk: {risk}")
print(f"  Decision: {note}")
print("  Audit: decision recorded; no production action executed.")
```

19.2 Actual Execution Output Screenshot

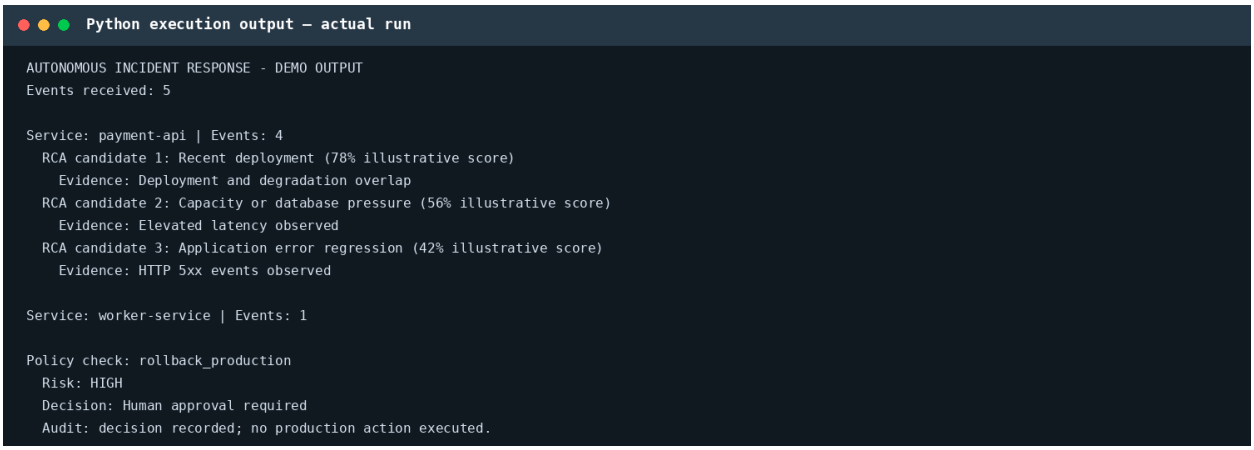


Figure 9. Captured console output from executing incident_analysis_demo.py.

The screenshot is generated from the actual output captured when the Python program was run during document preparation. It demonstrates sample grouping, candidate ranking and a high-impact action approval gate. It is not a live connection to a monitoring product.

19.3 Interpretation of Output

Output	Interpretation
Events received	Five sample events were processed.
Service grouping	Events grouped by service identifier.
RCA candidates	Rules rank candidates when deployment and degradation signals overlap.
Risk HIGH	Production rollback is high impact in this demo.
Approval required	No real rollback is executed by the sample program.
Audit message	Production system must persist a durable audit record.

20. Limitations, Ethics and Future Enhancements

20.1 Limitations

- The demo uses small manually prepared events and cannot establish real-world RCA accuracy.
- Rule-based scores are illustrative and not calibrated probabilities.
- The code does not connect to monitoring, ticketing, cloud or IaC systems.
- The screenshot is real demo output, not evidence of production deployment.
- Real integrations need authentication, secrets management, rate limits and contract tests.

20.2 Ethical and Operational Considerations

- Operators should see why a cause/action is recommended.
- RBAC and approval reduce the risk of compromised accounts making unrestricted changes.
- Historical data may contain confidential information and needs access and retention controls.
- False positives and false negatives should both be measured.
- Operators need a kill switch and a manual-response fallback.

20.3 Future Enhancements

Enhancement	Benefit	Prerequisite
Live monitoring connectors	Live rather than sample events	Approved API access
Dependency graph	Better impact analysis	Maintained service catalog
Historical incident search	Similar-case context	Clean incident history
Calibrated ML ranking	Potentially better ranking	Reviewed labelled examples
ChatOps	Faster collaboration	Identity/approval integration
Canary rollback	Controlled recovery	Test environment and release policy
Drift detection	Detect baseline changes	Reliable baseline metrics

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Thorough understanding of scenario and stakeholders.	Good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing.
Functional and Non-Functional Requirement Identification (25%)	Comprehensive and well-classified requirements.	Most requirements identified.	Basic requirements; omissions.	Incomplete or poorly classified.
Actor Identification and Use Case Modelling (20%)	All actors/use cases; accurate UML.	Most actors/use cases.	Limited actors/use cases.	Missing or inaccurate.
SRS Documentation (15%)	Complete, professional SRS.	Organized with minor gaps.	Essential sections with limited detail.	Incomplete or poorly organized.
Assumptions, Constraints and Validation (10%)	Realistic assumptions and strong validation.	Most covered adequately.	Limited or superficial.	Missing or incorrect.

Criteria	Mark
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirements (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
SRS Documentation (15%)	/5
Assumptions, Constraints and Validation (10%)	/5
TOTAL	/25